

Report delle modifiche al progetto RT-KalmanNet

1. KNet/RT_KalmanNet_nn.py

Architettura originale (prima delle modifiche)

La rete usava un `nn.ModuleList()` con un ciclo `for` esplicito che concatenava l'output del passo precedente (`self.previous_output`) all'input corrente ad ogni step. L'attivazione finale era `torch.sigmoid`, e il forward restituiva solo `c_t` come scalare senza restituire lo stato interno.

Cosa è cambiato e perché

1. `nn.ModuleList` → `nn.GRU`

Ispirazione: paper KalmanNet (Revach et al., 2022) e RT-KNet. La GRU è la cella ricorrente standard per modellare dipendenze temporali lunghe con gate di reset e aggiornamento, senza il vanishing gradient problem del RNN vanilla.

Il ciclo manuale con `ModuleList` replicava la logica di una RNN elementare ma in modo non differenziabile end-to-end (il `.detach()` sull'output precedente tagliava il grafo). La `nn.GRU` di PyTorch gestisce tutto internamente e propaga i gradienti correttamente.

```
self.gru = nn.GRU(
    input_size = gru_hidden_size,
    hidden_size = gru_hidden_size,
    batch_first = True
)
```

2. Stato nascosto (`h_t`) rimosso dalla classe e passato esternamente

Motivo: il `self.previous_output = torch.zeros(...)` fisso nella classe legava la memoria a un'unica sequenza. Con BPTT su mini-batch, ogni sequenza deve iniziare con `h_t = None` e lo stato deve fluire step-by-step all'interno del loop del filtro. Passare `h_t` fuori dalla rete permette al chiamante (`fnREKF`) di controllare esattamente quando la memoria viene resettata.

3. Forward: firma (`x, h_t=None`) → `return (c_t, h_t_nuovo)`

Il forward restituisce ora la tupla (`c_t, h_t`) per permettere il passaggio della memoria da uno step temporale al successivo dentro il loop del filtro.

4. Rimozione del `.detach()` sull'output

L'originale aveva `final_output.clone().detach().requires_grad_(False)`. Questo distruggeva il grafo computazionale e rendeva BPTT impossibile: il gradiente si fermava ad

ogni step. Rimosso completamente.

5. `torch.sigmoid` → `torch.nn.functional.softplus` sull'output

Motivo: `sigmoid` mappa in $(0,1)$, ma il parametro c nella REKF non ha un limite superiore fisico a 1. `softplus` è definita come $\log(1 + e^x)$, è smooth, positiva per ogni input, e non satura superiormente. Questo permette alla rete di esprimere valori di tolleranza anche > 1 .

6. Layer FC lineare sull'input (no ReLU)

Ispirazione: paper KalmanNet Fig. 3, che descrive i layer FC come "fully connected linear" (nessuna attivazione intermedia). Il layer `fcl_in` è quindi pura proiezione lineare da `input_size_fcl` a `gru_hidden_size`:

```
x_feat = self.fcl_in(x)  # linear, no activation
```

Un precedente tentativo con `torch.relu(self.fcl_in(x))` è stato rimosso per allinearsi alla specifica del paper.

7. Gestione dimensioni per la GRU

La GRU si aspetta input di forma `(batch, seq, features)`. Siccome il filtro processa un timestep alla volta dentro un loop, il tensore viene forzato a `(1, 1, gru_hidden_size)` prima di entrare nella GRU:

```
if x_feat.dim() == 1:
    x_feat = x_feat.unsqueeze(0).unsqueeze(0)
```

2. RobustKalmanPY/robust_kalman.py

fnComputeTheta — evoluzione in 5 versioni

Il problema è risolvere implicitamente:

$$g(\theta, c) = \text{tr}[(I - \theta P)^{-1} - I] + \log \det(I - \theta P) - c = 0$$

per trovare θ_t dato c_t (output della GRU) e P_t (covarianza predetta del filtro). La soluzione deve essere *differenziabile rispetto a c* per permettere la backpropagation verso la GRU.

V1 e V2 (commentate): bisection con `while` → rischio loop infinito. V2 aggiunge `max_iter=100` e un `break` su intervallo troppo piccolo. Non differenziabile rispetto a c .

V3 (commentata): `theta_max * torch.sigmoid(c_val)` — mappa smooth e differenziabile, ma non risolve l'equazione implicita: è una proxy che non ha significato fisico esatto.

V4 (commentata): primo tentativo di *implicit layer* completo: bisection in `torch.no_grad()` + un passo di Newton per il gradiente. Bug: il Newton step era fuori da qualsiasi contesto `enable_grad`, quindi dentro `torch.no_grad()` (loop CV) crashava con `element 0 of tensors does not require grad`.

V5 (versione attuale):

Ispirazione: teoria degli implicit layers (Amos & Kolter, DEQ, Bai et al.) e implicit function theorem (IFT). La derivata $d\theta/dc$ si ricava analiticamente dalla condizione $g(\theta, c) = 0$:

$$\frac{d\theta}{dc} = -\frac{\partial g / \partial c}{\partial g / \partial \theta} = \frac{1}{\partial g / \partial \theta}$$

Forward (root-finding numerico, senza grafo):

```
with torch.no_grad():
    # 50 iterazioni di bisezione
    # usa eigvalsh (più stabile di eig su matrici simmetriche)
    # usa slogdet invece di log(det(...)) per stabilità numerica
```

Backward (differenziabile, solo in training):

```
if torch.is_grad_enabled():
    theta = theta_hat.detach().clone().requires_grad_(True)
    g_val = g_fn(theta, c_val) # c_val porta il grad_fn della GRU
    dg_dtheta = torch.autograd.grad(g_val, theta, create_graph=True)[0]
    theta_refined = theta - g_val / dg_safe # Newton step
```

Il check `torch.is_grad_enabled()` è la chiave: in valutazione/CV (with `torch.no_grad():`) si restituisce `theta_hat` direttamente, evitando la chiamata a `autograd.grad` che crashava. In training si esegue il passo Newton che porta i gradienti verso c_t e quindi verso la GRU.

Altre migliorie in `fnComputeTheta`:

- `P = 0.5*(P_pred + P_pred.T)` per simmetrizzare esplicitamente la matrice
- `torch.linalg.eigvalsh` invece di `eigvals` (più stabile su matrici simmetriche reali)
- `torch.linalg.slogdet` invece di `log(det(...))` per evitare underflow/overflow
- `torch.clamp` su `c_val` e su `theta_refined` per mantenere le variabili nel dominio valido

fnREKF — riscrittura per BPTT

Problema originale: l'implementazione vecchia scriveva direttamente su `self.Xrekf[:, i+1]` (operazione in-place su buffer persistente), e le variabili di stato come `self.V_prev`, `self.Xrekf_prev` trattenevano riferimenti al grafo computazionale della chiamata

precedente. Al secondo epoch, il backward trovava un grafo già liberato → `RuntimeError: Trying to backward through the graph a second time.`

Soluzione: ogni chiamata inizia con una copia detached dello stato persistente:

```
x_prev = self.Xrekf_prev.detach().clone()
V_prev = self.V_prev.detach().clone()
```

Questo "taglia" il filo tra la chiamata attuale e quella precedente. Il grafo computazionale è fresco ad ogni sequenza. Al termine, le variabili persistenti vengono aggiornate sempre con `.detach()`:

```
self.Xrekf_prev = x_prev.detach()
self.V_prev = V_prev.detach()
```

L'output `Xrekf_out` viene costruito accumulando gli stati in una lista `x_seq` e concatenando con `torch.stack(x_seq, dim=1)` alla fine, evitando scritture in-place.

Integrazione della GRU:

```
h_t = None # reset prima del loop
...
input_features = input_features.view(1, 1, -1) # (batch=1, seq=1, feat)
self.c, h_t = self.nn(input_features, h_t) # la memoria scorre
step-by-step
```

Rimosso `.item()` su `self.c`: usarlo convertiva il tensore in Python scalar, distruggendo il grafo e rendendo impossibile il backprop verso la GRU.

3. `main_Robust_KNet.ipynb`

Cella Settings

Aggiunti parametri ridotti per test rapidi (`N_E=10`, `N_CV=5`, `T=50`, ecc.) con i valori full commentati sopra.

Cella REKF test

Riscritta per iterare su (`test_input_batch`, `test_target_batch`) e poi sul `seq_idx` interno al batch, siccome `test_inputs` è una lista di tensori `[N_T, p, T]`.

Cella plot REKF

Aggiunti `.detach()` prima di `.numpy()` su `Xrekf`: necessario perché l'output di `fnREKF` in modalità `use_nn=False` non richiede grad ma era passato attraverso `torch.linalg.solve` che può produrre tensori con `grad_fn`.

Cella istanziazione RT-KNet

Aggiunto parametro `gru_hidden_size=64` e `input_feat_mode=3` (usa tutte e 4 le feature: F_1, F_2, F_3, F_4).

Pre-generazione dataset

I dati di training e CV vengono generati **una volta per epoch prima del loop di training**, evitando la chiamata a `DataGen` dentro il loop stesso:

```
for epoch in range(epochs):
    DataGen(...)
    train_inputs.append(train_input)
    cv_inputs.append(cv_input)
```

Training loop — riscrittura completa

Ispirazione: standard BPTT truncated per sequence models (Werbos 1990, Williams & Zipser 1995), applicato come in KalmanNet (Revach et al.).

```
for epoch in epochs:
    zero_grad()
    campiona n_batch indici casuali
    for idx in batch_indices:
        resetta stato filtro (detached)
        fnREKF(train=True) → Xrekf
        loss = MSE(Xrekf, target)
        batch_losses.append(loss)
    avg_loss = stack(batch_losses).mean()
    avg_loss.backward()    # unico backward sull'intero mini-batch
    optimizer.step()
```

Punti chiave:

- `optimizer.zero_grad(set_to_none=True)` a inizio epoch, non dentro il loop delle sequenze
- Il reset esplicito dello stato filtro (`Xrekf_prev`, `V_prev`, `y_prev`, `Xn_prev`) prima di ogni sequenza assicura che i grafi non si mescolino tra sequenze
- Un singolo `backward()` sull'average loss è sufficiente; chiamarlo N volte (una per sequenza) avrebbe richiesto `retain_graph=True` e avrebbe accumulato gradienti in modo errato

CV loop

Dentro `with torch.no_grad():`, itera su tutte le sequenze CV, accumula le loss come `.item()` (nessun grafo), sceglie il best model su `opt_MSE`, salva con `torch.save(state_dict)`.

Test RT-KNet

Itera su tutti i 100 set di test (stessa struttura del REKF test), reset stato ad ogni sequenza, `Xrekf[:, :-1].detach()` prima di passare a MSE/plot.

Cella plot c_t

```
c_t_np = torch.stack([ct.detach() for ct in  
c_t]).cpu().numpy().reshape(-1)
```

`c_t` è una lista di tensori scalari (uno per timestep), `.detach()` necessario perché durante il test `c_t` viene generato con `nn.eval()` ma i tensori potrebbero comunque avere `grad_fn` residuo dalla computazione di `fnComputeTheta`.

Cella KalmanNet test

Aggiunto loop per iterare sequenza per sequenza dai batch di test (stesso schema degli altri test), usando `.unsqueeze(0)` per adattare la forma a quanto atteso da `NNTest`.